



JAVA PROGRAMMING

(340)

STATE 2023

PRODUCTION:

Crypto Exchange Portfolio Manager _____ (420 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

Note to Graders:

- Output will be static for the data in each of the fields per the provided text files. All formatting should be present in the printing to console.
- NOTE: we will not be checking for errors with bad sample files due to the complexity and length required. Try/Catch exceptions will be manually inspected for the three instances it should be present (see rubric).
- Ignore line spacing differences or grammar errors in the Field tags such Name, Quantity, etc. Some IDE's consoles will automatically wrap the text, please ignore these factors.
- *When reading code: you might consider using the State Solutions RTF file due to ease of readability compared to the embedded solution at the end of this document which is heavily affected by the template margins.*

Test Case #1: program executes flawlessly and displays the data as displayed below:

=====POSITIONS=====

Name: Bitcoin | Symbol: BTC | Quantity: 0.0005 | Current Price: \$48,912.0000

Average Price \$56,234.0000 | Purchase Price: \$59,343.0000 | Percent Change: -17.58%

Web URL: bitcoin.org | SourceCode URL: github.com/bitcoin | Market Cap: \$896,690,280,785 | Tag: PoW

Name: Ethereum | Symbol: ETH | Quantity: 2.3450 | Current Price: \$4,323.0000

Average Price \$4,465.0000 | Purchase Price: \$3,738.0000 | Percent Change: 15.65%

Web URL: Ethereum.org | SourceCode URL: https://github.com/ethereum | Market Cap: \$453,972,230,585 | Tag: PoW

Name: Polygon | Symbol: MATIC | Quantity: 56.3400 | Current Price: \$2.3400

Average Price \$1.8400 | Purchase Price: \$0.3400 | Percent Change: 588.24%

Web URL: https://polygon.technology/ | SourceCode URL: https://github.com/maticnetwork/ | Market Cap: \$12,920,349,074 | Tag: BEP20

Name: Sandbox | Symbol: SAND | Quantity: 1,222.0000 | Current Price: \$5.8700

Average Price \$2.7700 | Purchase Price: \$2.7200 | Percent Change: 115.81%

Web URL: www.sandbox.game/en/ | SourceCode URL: NA | Market Cap: \$4,498,404,032 | Tag: ERC-20

Name: Cardano | Symbol: ADA | Quantity: 3,245.0000 | Current Price: \$1.3400

Average Price \$2.2100 | Purchase Price: \$2.0900 | Percent Change: -35.89%

Web URL: cardano.org/ | SourceCode URL: <https://cardanoupdates.com/> | Market Cap: \$41,051,193,159
| Tag: DPOS

Name: Shiba-Inu | Symbol: SHIB | Quantity: 1,345,987.0000 | Current Price: \$0.0000

Average Price \$0.0001 | Purchase Price: \$0.0000 | Percent Change: 44.47%

Web URL: shibatoken.com/ | SourceCode URL: NA | Market Cap: \$18,256,090,722 | Tag: ERC-20

Name: Dogecoin | Symbol: DOGE | Quantity: 33,922.0000 | Current Price: \$0.1595

Average Price \$0.2343 | Purchase Price: \$0.2386 | Percent Change: -33.15%

Web URL: dogecoin.com/ | SourceCode URL: <https://github.com/dogecoin/dogecoin> | Market Cap: \$20,943,279,561 | Tag: BEP20

Name: Uniswap | Symbol: UNI | Quantity: 234.2000 | Current Price: \$14.7500

Average Price \$11.2300 | Purchase Price: \$5.8200 | Percent Change: 153.44%

Web URL: uniswap.org/blog/uni/ | SourceCode URL: NA | Market Cap: \$9,153,029,112 | Tag: ERC-20

Solution and Project		
The project is present on the flash drive	_____	10 points
The projects main class is named CryptoExchangePortfolioManager	_____	10 points
The class helper method is named readCoinTextFiles(BufferedReader coinsIO)	_____	10 points
The class helper method is named readBlockTextFiles(BufferedReader blockIO)	_____	10 points
The class helper method is named BufferedReader getReader(String name)	_____	10 points
The Position object helper method is named getInfo() and returns a string value that returns the following: (1)Name, (2) Symbol, (3) Quantity, (4)Current Price (5) Average Price, (6) Purchase Price, (7) Percent Change AND the information from the BlockInfo object (8)Web URL, (9) SourceCode URL, (10) Market Cap, (11)Tag	_____	10 points
The Position object helper method is named percentChange() calculates the percent of change from purchase price to the current price. Hint here is one possible way: $((\text{currentPrice} - \text{purchasePrice}) / \text{purchasePrice}) * 100$	_____	10 points
The BlockInfo object helper method is named getInfo() and returns a string value that returns the following: (8)Web URL, (9) SourceCode URL, (10) Market Cap, (11)Tag	_____	10 points
Program Execution		
The program runs from the USB flash drive	_____	10 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
The program displays the printout based upon the required order of investment position types per the instructions	_____	30 points
The output has each field in the records separated by a pipe symbol with no line starting or ending with the	_____	10 points
The output has a hanging indent for the second and third line.	_____	10 points
The Positions data for the output is the first and second line and is in the proper order: (1)Name, (2) Symbol, (3) Quantity, (4)Current Price (5) Average Price, (6) Purchase Price, (7) Percent Change	_____	10 points
The BlockInfo data for the output is the third line and is in the proper order: (8)Web URL, (9) SourceCode URL, (10) Market Cap, (11)Tag	_____	10 points
(3)Quantity is formatted for up to four decimal places	_____	10 points
(4)Current Price (5) Average Price, (6) Purchase Price is formatted for up to four decimal places, commas, and \$ symbol	_____	30 points
(10) Market Cap is formatted for with NO decimal places, and includes commas, and \$ symbol	_____	20 points

Source Code Review		
The source code is properly commented		
A comment containing the contestant number is present		10 points
Methods and code sections are commented		20 points
Method successfully uses try... catch for exception handling for BufferedReader getReader(String name) when creating the file based upon the passed parameter and returns the BufferedReader object to the coinsIO in the Main method. The error message should read: "The file doesn't exist."		30 points
Method successfully uses try... catch for exception handling for ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) to detect IOException errors; the error message should read "I/O Error Coin"		10 points
Method ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) extracts all data fields from text file (NOTE: double values must be parsed from String values)		10 points
Method ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) successfully calls BlockInfo readBlockTextFiles(BufferedReader blockIO) with proper text file based upon the symbol and stores the returned BlockInfo object. Be sure the readBlockTextFiles method has a try/catch exception with an the error message read "I/O Error Block"		50 points
Method ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO) successfully creates Positions object that is used as a parameter to construct an ExchangeWrapper object that is returned at the completion of the process.		40 points
In the Positions object class complete the String getInfo() method to print out the data fields from the object when called by returning a String in the proper order (remember items 8-11 are from the nested BlockInfo object): (1)Name, (2) Symbol, (3) Quantity, (4)Current Price (5) Average Price, (6) Purchase Price, (7) Percent Change (8)Web URL, (9) SourceCode URL, (10) Market Cap, (11)Tag		30 points
Total Points		/420 points

SUGGESTED SOLUTION

```

1
2 import java.util.*;
3 import java.io.*;
4 import java.util.Locale;
5
6
7 public class CryptoExchangePortfolioManager
8 {
9
10 //GIVEN
11     public static void main (String args [])
12     {
13
14
15         BufferedReader coinsIO = getReader("Coins.txt");
16         List<ExchangeWrapper> cryptoPortfolio = new ArrayList<>();
17         ExchangeWrapper wrappedPosition = readCoinTextFiles(coinsIO);
18         cryptoPortfolio.add(wrappedPosition);
19         System.out.println("=====POSITIONS=====");
20         while (wrappedPosition != null)
21         {
22             wrappedPosition = readCoinTextFiles(coinsIO);
23             if(wrappedPosition != null)
24                 cryptoPortfolio.add(wrappedPosition);
25         }
26         for (ExchangeWrapper c : cryptoPortfolio){
27             System.out.println(c.getInfo());
28         }
29
30
31     }
32
33     //!!STUDENT WILL MAKE THIS <=====
34     /*Reads the information from the file and breaks it into each of the
35     objects that go into the ExchangeWrapper object*/
36     private static ExchangeWrapper readCoinTextFiles(BufferedReader coinsIO)
37     {
38         //Positions Class plus Fields
39         Positions pos;
40         String type;
41         String name;
42         String symbol;
43         double quantity;
44         double currentPrice;
45         double avgCost;
46         double buyPrice;
47         BlockInfo blkInfo;
48
49         String line = "";
50         String[] data;
51
52
53         //catches errors in the read
54         try
55         {

```

```

56         line = coinsIO.readLine();
57     }
58     catch (IOException e)
59     {
60         System.out.println("I/O Error Coins");
61         System.exit(0);
62     }
63     //ends the read once it gets to the end
64     if (line == null)
65         return null;
66     else
67     {
68         data = line.split(";"); //";" is the delimiter to separate the fields
for the records
69         type = data[0];
70         name = data[1];    //each field goes in escalating arithmetic sequence
0 to 6
71         symbol = data[2];
72         quantity = Double.parseDouble(data[3]);
73         currentPrice = Double.parseDouble(data[4]);
74         avgCost = Double.parseDouble(data[5]);
75         buyPrice = Double.parseDouble(data[6]);
76
77         /*Construct the objects to be placed in the Positions object;
78         the symbol will need to be trimmed */
79         String temp = symbol+".txt";
80         temp = temp.trim();
81
82         //Read the Block Files to be constructed and added to the Positions
83         BufferedReader block = getReader(temp);
84         blkInfo = readBlockTextFiles(block);
85
86         pos = new Positions(type.trim(), name.trim(), symbol.trim(), quantity,
87                             currentPrice, avgCost, buyPrice, blkInfo);
88         return new ExchangeWrapper(pos);
89     }
90 }
91
92 //!!STUDENT WILL MAKE THIS <=====
93 /*Reads the information from the file and breaks it into each of the objects
that
94 go into the Positions object */
95 private static BlockInfo readBlockTextFiles(BufferedReader blockIO)
96 {
97     //Positions Class plus Fields
98     //The long value will need to be trimmed
99     BlockInfo blkInfo;
100     String web;
101     String source;
102     String trimHold;
103     long market;
104     String tag;
105
106     String line = "";
107     String[] data;
108

```

```
109
110     //catches errors in the read
111     try
112     {
113         line = blockIO.readLine();
114     }
115     catch (IOException e)
116     {
117         System.out.println("I/O Error Block");
118         System.exit(0);
119     }
120     //ends the read once it gets to the end
121     if (line == null)
122         return null;
123     else
124     {
125         data = line.split(";"); //"," is the delimiter to separate the fields
for the records
126         web = data[0];
127         source = data[1];    //each field goes in escalating arithmetic
sequence 0 to 3
128         trimHold = data[2];
129         market = Long.parseLong(trimHold.trim());
130         tag = data[3];
131
132         blkInfo = new BlockInfo(web, source, market, tag);
133         return blkInfo;
134     }
135 }
136
137 //!!STUDENT WILL MAKE THIS <=====
138 //imports the file properly for BufferedReader for reading all .txt files
139 private static BufferedReader getReader(String name)
140 {
141     BufferedReader coinsIO = null;
142     try
143     {
144         File file = new File(name);
145         coinsIO = new BufferedReader(
146             new FileReader(file) );
147     }
148     catch (FileNotFoundException e)
149     {
150         System.out.println(
151             "The file doesn't exist.");
152         System.exit(0);
153     }
154
155     return coinsIO;
156 }
157 }
158 }
159
160
161
162
```

```

163
164 /////// ExchangeWrapper Information: GIVEN
165 class ExchangeWrapper
166 {
167     public Positions positions;
168
169
170     public ExchangeWrapper(Positions p)
171     {
172         this.positions = p;
173
174     }
175
176     public String getInfo(){
177
178         return this.positions.getInfo();
179     }
180 }
181
182 /////// Positions: Partially Given
183 class Positions
184 {
185     private String type; //This is not used
186     private String name;
187     private String symbol;
188     private double quantity;
189     private double currentPrice;
190     private double avgCost;
191     private double buyPrice;
192     public BlockInfo blkInfo;
193     private double percentChange;
194
195
196     //GIVEN
197     public Positions(String t, String n, String s, double q, double cp, double
ac,
198
199         double bp, BlockInfo bi)
200     {
201         type = t;
202         name = n;
203         symbol = s;
204         quantity = q;
205         currentPrice = cp;
206         avgCost = ac;
207         buyPrice = bp;
208         blkInfo = bi;
209     }
210
211     //!!STUDENT WILL MAKE THIS <=====
212     public String getInfo()
213     {
214         return "\nName: " + name + " | Symbol: " + symbol + " | Quantity: " +
String.format("%.4f",quantity) +
215         "| Current Price: $" + String.format("%.4f",currentPrice) +
"\n\tAverage Price $" +
String.format("%.4f",avgCost) + " | Purchase Price: $" +

```

```

String.format("%.4f",buyPrice) +
216     " | Percent Change: " + percentChange() + "% \n\t" +
blkInfo.getInfo();
217     }
218
219     ///!!STUDENT WILL MAKE THIS <=====
220     private String percentChange(){
221         percentChange = ((currentPrice - buyPrice)/buyPrice) *100;
222         String temp = String.format("%.2f",percentChange);
223         return temp;
224     }
225 }
226
227 ///// BlockInfo: Partially Given
228 class BlockInfo
229 {
230     private String website;
231     private String sourceCode;
232     private long marketCap;
233     private String tag;
234
235     public BlockInfo(String w, String sc, long mc, String t)
236     {
237         website = w;
238         sourceCode = sc;
239         marketCap = mc;
240         tag = t;
241     }
242
243     ///!!STUDENT WILL MAKE THIS <=====
244     public String getInfo()
245     {
246         return "Web URL: " + website + " | SourceCode URL: " + sourceCode +
247             " | Market Cap: $" + String.format("%,d", marketCap) + " | Tag: " + tag;
248     }
249
250 }
251

```

